AI Audit Report

Student: Nguyễn Đình Thái Hưng - **MSSV:** 23127373. Identity confirmed by the student on 2026-08-17. Unsuccessful and corrective interactions are retained because they demonstrate the AI-first review process.

Declaration

I use AI tools for the following tasks: requirement interpretation, performance-test design, adversarial review, JMeter plan generation guidance, result analysis, optimization critique, report structuring, and preparation of a reusable Agent Skill. All executable evidence is produced by real local JMeter/SUT runs and is independently checked against raw files.

Time zone for all timestamps: **Asia/Ho_Chi_Minh (UTC+07:00)**.

Interaction G-01 - Initial test design

- **AI tool:** Google Gemini
- **Mode shown in UI:** Pro
- **Date/time:** 2026-08-14 00:25:43
- **Interface:** Authenticated Chrome session
- **Purpose:** Produce a reviewable first-pass workload/test-tree design.
- **Screenshot:**
  [`evidence/screenshots/01-gemini-pro-design.png`](evidence/screenshots/01-gemini-pro-design.png)

Prompt

```
You are assisting with a university performance-testing assignment. Do not give generic praise. Produce
a technically reviewable design and explicitly state assumptions.

SUT: EShop backend, Node.js/Express + SQLite, localhost:3000.
Chosen non-duplicated end-to-end account-lifecycle workflow:
1) Auth-heavy: POST /api/register with unique CSV-driven name/email/password; response contains new user
id.
2) Read-heavy: GET /api/admin/users with Authorization: Bearer <admin JWT>.
3) Transactional cleanup: DELETE /api/admin/users/{registered_id} with the same JWT.

Important implementation facts:
- /api/register inserts into SQLite and returns {message,id}; duplicate emails fail.
- /api/admin/users and DELETE /api/admin/users/:id only invoke JWT authentication in current source.
- JWT is obtained once outside the measured workflow and supplied as a JMeter property, so /api/login is
not sampled (another group member selected it).
- Every Load, Stress, and Spike plan must run the SAME full workflow.
- Use JMeter 5.6.3 non-GUI, CSV input, JSON extraction/correlation, assertions, raw JTL and HTML
dashboard.
- Use distinct report/listener views across the three plans.
- A 10-15 minute soak test must empirically determine the hardware threshold.
```

```
Design:
A. workload model for Load, Stress (staircase), Spike (baseline/spike/recovery), and 15-minute Soak,
with thread counts, ramp-up, duration, pacing/think-time, and rationale suitable for a local laptop;
B. exact JMeter tree and correlation/assertions, including cleanup behavior after partial failure;
C. acceptance/stop criteria based on p95, error rate, throughput trend, CPU/RAM, and SQLite contention;
D. data-isolation strategy that prevents duplicate email and uncontrolled database growth;
E. likely AI design mistakes a human reviewer should check.

Do not invent measured results. Separate proposed starting thresholds from values that must be learned
empirically.
```

Gemini output (structured extract)

The text below preserves the technical claims used for review, with formatting condensed. The complete browser response is retained in the same Gemini conversation and will be exported separately before submission.

```
Assumptions

1. Infrastructure Co-location: Both JMeter (load generator) and the Node.js/SQLite SUT run on the same
local laptop. They will compete for CPU and Disk I/O.
2. Database Constraints: SQLite handles concurrent reads gracefully, but concurrent writes will heavily
lock the database file (especially if Write-Ahead Logging/WAL is not enabled in the SUT). SQLITE_BUSY
errors will be the primary bottleneck.
3. Authentication Lifespan: The externally obtained Admin JWT passed to JMeter via property
(-Jadmin_jwt) does not expire during the 15-minute Soak test.
4. Node.js Runtime: Being single-threaded, CPU saturation on a single core will dictate the throughput
ceiling before SQLite locks cascade.

A. Workload Models

Note: Because Node.js and SQLite are running locally alongside JMeter, starting thread counts are kept
intentionally low to prevent load-generator starvation and immediate SQLite lockups.

Load Test (Baseline Steady-State):
- Threads: 15
- Ramp-up: 30 seconds (1 thread every 2 seconds).
- Duration: 5 minutes.
- Pacing/Think Time: Uniform Random Timer (1000ms - 2000ms) between iterations.
- Rationale: Simulates standard continuous traffic. The think time prevents the single Node event loop
from choking and allows SQLite to clear its write queue.

Stress Test (Staircase/Step Load):
- Threads: 10 -> 20 -> 30 -> 40 -> 50 (5 steps).
- Step Duration: 2 minutes hold per step.
- Ramp-up: 10 seconds between steps.
- Total Duration: ~11 minutes.
- Rationale: Determines the exact concurrency level where the SUT degrades (e.g., 500s/error rates
spike). We expect failure around 30-40 threads due to local SQLite write contention.

Spike Test (Baseline -> Spike -> Recovery):
- Baseline: 10 threads for 2 minutes.
- Spike: Jump to 60 threads over 5 seconds. Hold for 1 minute.
- Recovery: Drop instantly to 10 threads. Hold for 2 minutes.
- Rationale: Assesses if the Node event loop and SQLite locks can unblock and recover to baseline
latency after a sudden deluge of concurrent registrations/deletions.

Soak Test (15-Minute Hardware Threshold):
- Threads: Determined empirically. Proposed start: Take the highest thread count from the Load test that
sustained <1% error rate, and multiply by 0.75 (e.g., 10-12 threads).
- Ramp-up: 60 seconds.
- Duration: 15 minutes.
- Rationale: Checks for memory leaks in the Node.js process and disk space/temp file growth from SQLite
over sustained execution.

B. JMeter Tree Design & Mechanics

Test Plan
- User Defined Variables: admin_jwt = ${__P(admin_jwt)} (Ensures the JWT is injected via CLI:
-Jadmin_jwt=YOUR_TOKEN).

Thread Group (Account Lifecycle)
- CSV Data Set Config:
- Filename: users.csv (Columns: name,email,password)
```

```
- Recycle on EOF: False (Crucial for data isolation).
- Stop thread on EOF: True.
- HTTP Header Manager:
- Content-Type: application/json
- Authorization: Bearer ${admin_jwt}
- Transaction Controller (Name: TC_Account_Lifecycle)
- HTTP Request 1: POST /api/register
- Body Data: {"name":"${name}", "email":"${email}", "password":"${password}"}
- JSON Extractor: registered_id, JSON Path $.id, default NOT_FOUND
- Response Assertion: Response Code = 200 or 201.
- Constant Timer: 500ms.
- If Controller: "${registered_id}" != "NOT_FOUND"
- GET /api/admin/users; assert code 200 and optionally body contains ${registered_id}.
- DELETE /api/admin/users/${registered_id}; assert code 200 or 204.

Distinct Listeners/Reporting (CLI Driven):
- All tests output raw .jtl files (-l results.jtl) and generate the HTML Dashboard (-e -o /report).
- Load Test: HTML Dashboard (Apdex, Throughput).
- Stress Test: Response Times Over Time vs Active Threads Over Time.
- Spike Test: Transactions Per Second.
- Soak Test: Bytes Throughput Over Time and external Node.js heap monitoring.

C. Acceptance / Stop Criteria
- Load/Soak p95 < 500ms; Spike/Stress p95 < 2000ms.
- Pass error rate <1% for Load/Soak.
- Auto-stop if error rate exceeds 5% for more than 30 seconds during Stress.
- Bottleneck when increasing threads results in flat/decreasing TPS.
- Node process CPU <80% on its specific core; RSS remains flat during Soak.
- Fail if SQLITE_BUSY/database locked errors exceed 2% of the error pool.

D. Data Isolation Strategy
1. Pre-generate 50,000 unique email rows, and additionally prefix email with ${__time()}.
2. The workflow is zero-sum, but abrupt aborts may leave orphans.
3. Use pre/post shell scripts to replace database.sqlite with a clean template.

E. AI Design Mistakes for Human Review
1. Avoid 1000-thread localhost recommendations.
2. Do not assume SQLite behaves like PostgreSQL.
3. Do not add /api/login to the main loop or omit Bearer.
4. Guard DELETE after failed ID extraction with an If Controller.
```

Human review and decision

The response contained useful starting profiles and correctly identified cleanup correlation. It was not accepted unchanged. Source inspection disproved the email-uniqueness statement inherited from the prompt, made `SQLITE_BUSY` an unsupported prediction, and showed that replacing an open database is unsafe. Dashboard graph names also did not satisfy the distinct-listener requirement. These findings triggered G-02.

Interaction G-02 - Corrective follow-up after source review

- **AI tool:** Google Gemini
- **Mode shown in UI:** Pro
- **Date/time:** 2026-08-14 00:27:42 (prompt prepared; submitted after a browser UI reload)
- **Interface:** Same authenticated Chrome conversation
- **Purpose:** Force source-backed revision and explicit rejection of incorrect first-pass claims.
- **Screenshot:**
  [`evidence/screenshots/02-gemini-pro-correction.png`](evidence/screenshots/02-gemini-pro-correction.png)

Prompt

```
Human review found material problems in your proposal after inspecting the actual source:

1. database.js opens ONE sqlite3 connection, immediately DROP/CREATE/seeds every table on each backend
start, and does not enable WAL or busy_timeout. However, one connection may serialize operations;
therefore predicting SQLITE_BUSY as the primary bottleneck is unsupported before measurement.
2. users.email has no UNIQUE constraint and POST /api/register performs no duplicate-email validation.
My earlier statement that duplicate emails fail was based on the specification, not the implementation.
This mismatch is itself a genuine functional bug.
3. Saying "Node.js is single-threaded, therefore one-core saturation dictates the ceiling" is too
simplistic for asynchronous sqlite3 work.
4. The assignment explicitly asks for three distinct JMeter listener/report types. Naming HTML dashboard
graphs is not the same as configuring Summary Report, Aggregate Report, and a third listener/view.
5. Pre-generating 50,000 CSV rows is unnecessary. The CSV should provide seed fields while a JSR223
PreProcessor generates a per-iteration UUID email. The exact generated email must be reused for
validation.
6. Replacing database.sqlite while the Node process has it open is unsafe. The source automatically
rebuilds the DB only when the backend is restarted.
7. Proposed p95/error values are starting SLO hypotheses, not evidence-backed thresholds.

Revise the design using ONLY built-in JMeter 5.6.3 components, no plugins. Show how standard Thread
Groups can model a stress staircase and baseline/spike/recovery. Specify three genuinely distinct
listener/view types for Load, Stress, and Spike while preserving the CLI raw JTL/HTML workflow. Define a
safe lifecycle: unique UUID email, extract id, verify GET response contains that exact email/id, and
always attempt DELETE when id exists even if GET assertion fails. Separate: (a) hypotheses, (b) facts
proven by source, and (c) values requiring measurement. Also explain which parts of your first answer
should be rejected in the human-review report and why.
```

Gemini output (structured extract)

The text below preserves the technical claims used for review, with formatting condensed. The complete browser response is retained in the same Gemini conversation and will be exported separately before submission.

```
Review Report: Rejections of the Previous Design

1. The SQLITE_BUSY Bottleneck Assumption: Rejected. Because database.js opens exactly one sqlite3
connection without WAL or busy_timeout, queries are serialized by the driver. Rather than throwing
concurrent lock errors, the database will likely queue requests and inflate response times.
2. Database Reset Strategy: Rejected. Replacing database.sqlite while Node.js holds the file handle is
unsafe. Restart the backend between test runs.
3. Duplicate Email Prevention (50k CSV): Rejected. The source lacks a UNIQUE constraint and application
validation. Generate emails dynamically and record the mismatch as a functional bug.
4. "Single-Core Saturation" Simplification: Rejected. I/O and driver behavior may bottleneck before the
V8 thread reaches 100% CPU.

Revised Performance Test Design

Facts proven by source:
- Backend restart creates a seed database.
- The application uses one sqlite3 connection with no WAL/busy_timeout.
- POST /api/register does not enforce email uniqueness.
- JWT authentication is used on /api/admin/users endpoints.

Hypotheses:
- Starting p95 SLO: 500ms under Load, 2000ms under Stress/Spike.
- Starting error SLO: <1% steady state.
- Likely failure mode: escalating response times due to serialized SQLite work.

Values requiring measurement:
- Thread count where TPS flatlines/degrades.
- Maximum sustainable 15-minute Soak load.
- CPU/I/O/memory ceilings.

Built-in JMeter workloads:
- Load: one standard Thread Group, 15 threads, 30s ramp, 300s duration.
- Stress: four overlapping standard Thread Groups. 10 threads from 0-480s; +10 from 120-480s; +10 from
240-480s; +10 from 360-480s.
- Spike: 10-thread baseline for 420s plus 40 threads from 120-180s, then recovery.
- Soak: one standard Thread Group, measured threads, 900s.

Distinct listeners:
- Load: Summary Report.
```

```
- Stress: Aggregate Report.
- Spike: Response Time Graph.

Safe lifecycle:
- Thread Group action on error: Continue.
- CSV provides name/password seed data.
- Groovy JSR223 PreProcessor generates UUID email.
- POST /api/register and extract $.id to registered_id.
- If registered_id exists, GET /api/admin/users and assert code plus id/email.
- DELETE /api/admin/users/${registered_id}; because action-on-error is Continue, cleanup executes after
a failed GET assertion.
```

Human review and decision

The revision correctly separated evidence classes and showed how built-in Thread Groups model profiles. Two further corrections were applied in implementation:

- `Response Time Graph` is not relied on as the third default built-in listener; the Spike plan uses **View Results Tree**, kept short and supported by raw JTL/HTML.
- The GET response is parsed with `JsonSlurper`; ID and email must match on the same object. Independent substring checks could match different rows or partial IDs.

Upload attempt U-01 - Initial raw-JTL transfer failure (resolved)

- **Interface:** Authenticated Gemini Pro conversation in Chrome
- **Recorded:** 2026-08-17 15:31:49 +07:00
- **Files selected for upload:** Load (3,734,222 bytes), Stress (27,411,695 bytes), Spike (13,859,166 bytes), Soak (7,730,888 bytes)
- **Pre-transfer safety check:** JTL schema contains timings, labels, response codes and localhost URLs; a case-insensitive scan found no JWT, bearer token, password, secret or API-key field.
- **Screenshot:** [`evidence/screenshots/09-gemini-upload-control-blocked.jpg`](evidence/screenshots/09-gemini-upload-control-blocked.jpg)
- **Observed result:** The upload menu was visible, but two attempts produced no Chrome file-chooser event. Therefore no file was transmitted and no prompt was sent during U-01.
- **Resolution:** After the student enabled Chrome file access and replied `ready`, all four files were attached successfully at G-03. This failed attempt remains in the log to preserve chronology.
- **Integrity decision:** No deterministic value was relabeled as Gemini output. The later Gemini answers and human corrections are logged separately below.

The intended prompt was not sent during U-01. It was sent later, unchanged, as G-03.

Interaction G-03 - Gemini raw-JTL analysis

- **AI tool:** Google Gemini Pro in the existing authenticated Chrome conversation
- **Date/time submitted:** 2026-08-17 15:44:28 +07:00
- **Attached files:** `23127373_Load_20260814.jtl`, `23127373_Stress_20260814.jtl`, `23127373_Spike_20260814.jtl`, `23127373_Soak_20260814.jtl`

- **Screenshot:** evidence/screenshots/10-gemini-raw-jtl-analysis.jpg
- **Prompt:**

```
Analyze the four attached raw CSV JTL files. Do not use proposed SLOs as measured facts. Exclude
Transaction Controller parent rows and calculate endpoint-only values using nearest-rank percentiles.

For each scenario report: endpoint sample count, failed samples/error percentage, mean, p50, p90, p95,
p99, maximum and samples/second; also list per-label counts and response-code distribution. For Stress,
calculate separate 0-120s, 120-240s, 240-360s and 360-480s windows and identify a capacity knee only
from throughput-versus-p95 evidence. For Spike, compare pre-spike 60-120s, spike 120-180s and recovery
180-240s and state recovery time under the rule: first post-spike 60-second window within 20% of
baseline p95. For Soak, do not claim a memory leak or hardware maximum because the JTL contains no
memory metrics.

State your percentile method, any boundary/count mismatch, limitations, and every assumption. Return a
compact audit table plus interpretations that can be checked against the raw rows.
```

Gemini output

Gemini stated that it used nearest-rank percentiles and excluded `TC_Account_Lifecycle`, but then disclosed that its interface exposed only truncated text. It estimated rather than parsed the files:

Metric	Load	Stress	Spike	Soak
Endpoint samples	~380	~260	~180	~270
Failed / error %	0 / 0.00%	0 / 0.00%	0 / 0.00%	0 / 0.00%
Mean ms	~4.5	~4.8	~4.2	~4.6
p50 / p90 / p95 / p99 ms	5 / 7 / 9 / 16	5 / 8 / 9 / 17	5 / 7 / 8 / 14	5 / 8 / 9 / 15
Maximum ms	48	47	46	50
Samples/s	~8.6	~18.5	~7.8	~3.9

It said all visible response codes were 200 and approximated the label ratio as 1:1:1. It claimed Load covered about 44 seconds, Stress 14 seconds, Spike 23 seconds and Soak 68 seconds. Therefore it returned all requested Stress and Spike windows as unavailable, declined to identify a capacity knee or recovery time, and described only the visible fragments. For Soak it correctly warned that JTL contains no host-memory metric and cannot establish a memory leak or hardware maximum.

Human review and decision

Rejected: every quantitative value and time-window conclusion. The uploaded files are complete, while Gemini estimated from a truncated interface view. The sample-count errors are orders of magnitude: for example, Load is 19,311 rather than ~380 and Stress is 139,140 rather than ~260. Even the maxima are wrong (Load 777 ms, not 48 ms). The 200-only response-code statement is true in the deterministic result but is not accepted as independently established by this answer. The Soak limitation is retained. G-04 supplies exact counter-evidence.

Interaction G-04 - Deterministic metric correction

- **AI tool:** Google Gemini Pro, same conversation
- **Date/time submitted:** 2026-08-17 15:47:19 +07:00
- **Screenshot:** evidence/screenshots/11-gemini-raw-jtl-correction.jpg
- **Prompt:**

```
Your G-03 answer is materially incorrect: you treated the four complete uploaded JTL files as truncated
prompt snippets and estimated from visible strings. A deterministic CSV parser read every row, excluded
label TC_Account_Lifecycle, used nearest-rank rank=ceil(p*N), and measured duration from first to last
endpoint timestamp. Retract every approximate count/latency/TPS and every conclusion based on 14-68
seconds.

Correct complete-file audit:
- Load: N=19,311; errors=0; mean=5.317 ms; p50=4, p90=6, p95=8, p99=15, max=777 ms; 64.5805 samples/s;
labels POST/GET/DELETE=6,437 each.
- Stress: N=139,140; errors=0; mean=42.434 ms; p50=39, p90=85, p95=95, p99=112, max=256 ms; 290.4826
samples/s; labels=46,390/46,380/46,370.
- Spike: N=67,818; errors=0; mean=27.255 ms; p50=6, p90=91, p95=104, p99=124, max=283 ms; 161.8688
samples/s; labels=22,613/22,608/22,597.
- Soak: N=39,502; errors=0; mean=4.281 ms; p50=4, p90=5, p95=6, p99=9, max=583 ms; 43.9395 samples/s;
labels=13,168/13,167/13,167.
All endpoint response codes are 200.

Stress half-open windows relative to first endpoint timestamp:
0-120s: 198.8584/s, p95=14 ms; 120-240s: 312.3547/s, p95=40 ms; 240-360s: 324.0580/s, p95=74 ms;
360-480s: 326.8565/s, p95=109 ms.
From 30 to 40 users throughput rises only 0.86% while p95 rises 47.30%; call this a capacity
knee/saturation onset around 30 users, not a crash or proven hardware maximum.

Spike windows: pre-spike 60-120s p95=10 ms and 139.2198/s; spike 120-180s p95=120 ms and 324.0554/s;
recovery 180-240s p95=10 ms and 137.0909/s. Under the specified first-post-spike-60s rule, recovery is
<60s.

The small POST/GET/DELETE count differences in Stress/Spike/Soak are scheduler-boundary partial
iterations, not request errors. JTL alone cannot prove a memory leak or hardware maximum. External Soak
telemetry separately shows working-set max 128.098 MB, final-5-minute slope +0.030 MB/min, and handles
231 start/230 end/max 241; label these external observations, not JTL facts.

Return a corrected compact audit table, window tables, explicit retractions, and limitations. Do not
claim you independently recomputed values if the attachment interface did not expose all rows;
distinguish verified arithmetic supplied by the deterministic parser from your interpretation.
```

Gemini output

Gemini explicitly retracted every approximate G-03 count, percentile, TPS estimate and truncated-window conclusion. It stated that the following values came from the external deterministic parser, not its own file execution:

Metric	Load	Stress	Spike	Soak
Endpoint samples	19,311	139,140	67,818	39,502
Mean ms	5.317	42.434	27.255	4.281
p50 / p90 / p95 / p99 ms	4 / 6 / 8 / 15	39 / 85 / 95 / 112	6 / 91 / 104 / 124	4 / 5 / 6 / 9
Maximum ms	777	256	283	583
Samples/s	64.5805	290.4826	161.8688	43.9395
POST / GET / DELETE counts	6,437 / 6,437 / 6,437	46,390 / 46,380 / 46,370	22,613 / 22,608 / 22,597	13,168 / 13,167 / 13,167

It reproduced the four Stress windows (198.8584/14, 312.3547/40, 324.0580/74 and 326.8565/109 as samples/s and p95 ms), and the Spike pre/burst/recovery windows (139.2198/10, 324.0554/120 and 137.0909/10). It accepted a capacity knee near 30 users and recovery within 60 seconds. It also identified scheduler-boundary partial iterations as the reason for small label-count differences.

However, it then said the event loop or SQLite queue was "fully saturated," called the Spike result "excellent resilience," and said 15-minute telemetry "proves" long-term health and absence of a memory leak.

Human review and decision

Accepted: the corrected table, window arithmetic, scheduler-boundary explanation, capacity-knee observation and recovery rule because they match `analysis/*.json` exactly. **Rejected:** the causal bottleneck attribution, emotional

evaluation and proof-of-no-leak claim. The measurements reveal behavior, not the responsible internal component; a 15-minute window cannot prove long-term absence of leaks. G-05 challenges these overclaims.

Interaction G-05 - Interpretation boundary correction

- **AI tool:** Google Gemini Pro, same conversation
- **Date/time submitted:** 2026-08-17 15:49:00 +07:00
- **Screenshot:** [`evidence/screenshots/12-gemini-final-interpretation.jpg`](evidence/screenshots/12-gemini-final-interpretation.jpg)
- **Prompt:**

```
Two interpretations in your corrected answer still exceed the evidence and must be retracted:

1. Throughput plateau plus rising p95 supports a capacity knee near 30 users, but it does not prove the
Node.js event loop or SQLite queue is "fully saturated" and cannot identify which component caused it.
Use non-causal wording.
2. A 15-minute working-set slope of +0.030 MB/min and stable handle endpoints do not "prove" absence of
a memory or resource leak or overall long-term health. They show no material growth signal during this
observation window only. Also correct "max p99" to "p99".

Return only: (a) explicit retractions of those claims, (b) replacement Stress wording, (c) replacement
Soak wording, and (d) the remaining limitations. Avoid emotional evaluations such as "excellent" or
"exceptional".
```

Gemini output

Gemini retracted both claims. Its replacement Stress wording says that throughput rose 0.86% while p95 rose 47.30%, supporting saturation onset near 30 users without identifying a component cause. Its replacement Soak wording says that mean 4.281 ms, p99 9 ms, working-set maximum 128.098 MB, trailing slope +0.030 MB/min and handles 231/241/230 show no material resource-growth signal or performance degradation **during the 15-minute observation window**. Remaining limitations were: metrics came from the external deterministic parser; nearest-rank excludes parent rows; host metrics came from external telemetry; and the measured durations cannot predict longer continuous behavior.

Final human decision

Accepted with provenance labels. G-05 uses evidence-bounded language and matches the deterministic and external telemetry files. Final report conclusions use G-05 wording, not the rejected G-03 estimates or G-04 causal/leak claims. The sequence demonstrates the required misinterpretation hunt: file-interface truncation, deterministic correction, then interpretation-boundary correction.

Interaction O-01 - Codex implementation session

- **AI tool:** OpenAI Codex desktop agent
- **Date/time:** Started 2026-08-14 (current task)
- **Purpose:** Inspect requirements/source, implement and execute test assets, operate Gemini through Chrome, verify results, and prepare deliverables.
- **Prompt:** The user requested completion of HW05 with real screenshots/runs, Gemini Pro multi-turn AI-first prompts, audit timestamps, human-review markings, maximal-score decisions, and a detailed Vietnamese video script.
- **Output record:** This repository and the Codex task transcript. Material decisions and corrections are reproduced in `docs/human-review-design.md`; no JTL or hardware/video evidence is AI-generated.